

GSTR-2A Reconciliation Tool

Developer Reference Document

Version 2.0 | gstr2a_app.py | ~1530 lines

Purpose of This Document

This document is the complete technical reference for `gstr2a_app.py`.

It covers: file structure, every function, constants, data flow,
how to add new features, known limitations, and how to build the `.exe`.

Written for the developer who built and maintains this tool.

1. Architecture Overview

The entire application lives in one file: `gstr2a_app.py`. It is structured into 12 logical sections separated by comments.

File Structure — 12 Sections

Section 1	Constants & Imports	— All global variables, colour palette, column alias dict
Section 2	Month Detection	— <code>detect_month_from_filename()</code> , <code>detect_month_from_data()</code>
Section 3	Data Helpers	— <code>clean_invoice()</code> , <code>clean_gstin()</code> , <code>_get_col()</code> , <code>_rename_cols()</code>
Section 4	File Loading	— <code>_find_header_row()</code> , <code>_extract_sheet()</code> , <code>load_gst_file()</code> , <code>load_pb_file()</code>
Section 5	Grouping (unused)	— <code>group_files_by_month()</code> — kept for future month-split feature
Section 6	Aggregation	— <code>aggregate()</code>
Section 7	GSTIN Validation	— <code>validate_gstins()</code>
Section 8	Reconciliation Engine	— <code>reconcile()</code>
Section 9	Excel Formatting Helpers	— <code>_fmt_ws()</code> , <code>_highlight_mismatch()</code> , <code>_totals_row()</code>
Section 10	Vendor Summary	— <code>build_vendor_summary()</code> , <code>_write_vendor_summary_sheet()</code>
Section 11	Cross Match	— <code>build_cross_match()</code> , <code>_write_cross_match_sheet()</code>
Section 12	Export + Summary Sheets	— <code>export_report()</code> , <code>_write_flat_summary()</code>
Section 13	Pipeline	— <code>run_reconciliation()</code>
Section 14	GUI	— <code>class App(tk.Tk)</code>

Technology stack:

- **Python 3.10+** — main language
- **pandas** — data loading, cleaning, aggregation, merging
- **openpyxl** — Excel file writing and formatting
- **tkinter** — GUI framework (built into Python, no install needed)
- **threading** — runs pipeline in background so GUI stays responsive
- **PyInstaller** — packages everything into a single .exe file

2. Data Flow — End to End

This is the complete journey of data from input files to output Excel:

Pipeline Steps

- Step 1 → User selects GST file(s) + PB file(s) + output folder via GUI
- Step 2 → `load_gst_file()` reads each GST file — scans ALL sheets
 `_find_header_row()` detects where headers start (row 0, 1, 2...)
 `_extract_sheet()` pulls 6 standard columns from each usable sheet

All sheets stacked into one combined GST DataFrame

Step 3 → `load_pb_file()` reads first sheet of each PB file

`_rename_cols()` maps raw column names → standard names via `COL_ALIASES`

Step 4 → All GST DataFrames `pd.concat()` into one

All PB DataFrames `pd.concat()` into one

Step 5 → `validate_gstins()` — flags invalid 15-char GSTINs (does NOT remove)

Step 6 → `aggregate()` — `groupby(GSTIN + InvoiceNo).sum()` on both sides

Step 7 → `reconcile()` — `pd.merge(outer)` → splits into 4 buckets

Step 8 → `build_vendor_summary()` + `build_cross_match()` — analysis tables

Step 9 → `export_report()` — writes all sheets to Excel with formatting

Step 10 → GUI shows badges + success popup

3. Constants Reference

Constant	Value / Purpose
OUTPUT_FILE	Final_Reco_Report.xlsx — name of the output file
KEY_COLS	["GSTIN", "Invoice Number"] — columns used as merge keys
VALUE_COLS	["Taxable Value", "IGST", "CGST", "SGST"] — numeric columns
MISMATCH_TOL	1.0 — if total tax diff > ₹1, invoice goes to Mismatched
GSTIN_RE	Regex: <code>^\d{2}[A-Z]{5}\d{4}[A-Z]\d[Z][A-Z\d]\$</code> — 15 char format
MONTHS	Full month name list — used for month detection and display
MONTH_ABBR	3-letter month abbreviations — used in filename detection
COL_ALIASES	Dict of 40+ column name variants → standard name mapping
C	GUI colour palette dict — all hex colours for tkinter widgets
SHEET_COLOURS	Dict of sheet name → hex colour for openpyxl header fills
THIN	openpyxl Border object — used on every Excel cell
RED_FILL	PatternFill — used to highlight mismatched cells
YELLOW_FILL	PatternFill — used for TOTAL rows
ORANGE_FILL	PatternFill — used for Duplicates sheet rows

To change the mismatch tolerance from ₹1 to ₹5:

```
MISMATCH_TOL = 5.0    # Change this one line
```

To add a new column name variant:

```
# Inside COL_ALIASES dict, add:
"my new column name" : "Taxable Value",    # maps to standard name
```

4. Function Reference

4.1 Data Loading Functions

Function	Input	Returns	Purpose
<code>_find_header_row()</code>	filepath, sheet_name, scan_rows=15	int (row index)	Scans first N rows to find which row has the actual column headers. Returns 0-based index.
<code>_extract_sheet()</code>	filepath, sheet_name	DataFrame None	Extracts 6 standard columns from one sheet. Returns None if sheet has no invoice data.
<code>load_gst_file()</code>	filepath, log	DataFrame	Loads one GST Portal file. Scans all sheets via <code>_extract_sheet()</code> , stacks usable ones.
<code>load_pb_file()</code>	filepath, log	DataFrame	Loads first sheet of one Purchase Book file. Maps columns via COL_ALIASES.

4.2 Cleaning Functions

Function	Input	Returns	Purpose
<code>clean_invoice(val)</code>	Any value	str	Strips all non-alphanumeric chars, uppercases. INV/001 → INV001. NaN → empty string.
<code>clean_gstin(val)</code>	Any value	str	Strips whitespace, uppercases. NaN → empty string.
<code>_get_col()</code>	df, col, default=0	Series	Safely fetches a numeric column. Missing column → Series of default value. Coerces to float.
<code>_get_str_col()</code>	df, col, default=""	Series	Safely fetches a string column. Missing column → Series of default value.
<code>_rename_cols(df)</code>	DataFrame	DataFrame	Applies COL_ALIASES to rename raw column

			names to standard names.
--	--	--	--------------------------

4.3 Core Pipeline Functions

Function	Input	Returns	Purpose
<code>aggregate()</code>	df, label, log	DataFrame	groupby(GSTIN+InvoiceNo).sum() on VALUE_COLS. Also picks worst ITC Eligibility per invoice.
<code>validate_gstins()</code>	df, label, log	list[str]	Checks all GSTINs against GSTIN_RE regex. Returns list of warning strings. Does NOT remove.
<code>reconcile()</code>	pb, gst, log	4 DataFrames	pd.merge(outer) on KEY_COLS. Splits into matched/mismatched/miss_gst/miss_pb.
<code>run_reconciliation()</code>	gst_files, pb_files, out_folder, log, set_progress	str (path)	Full pipeline — load, concat, validate, aggregate, reconcile, export. Returns output path.

4.4 Analysis Functions

Function	Input	Returns	Purpose
<code>build_vendor_summary()</code>	pb_agg, gst_agg	DataFrame	Groups by GSTIN. Sums PB values, GST values, calculates diffs. Merges vendor names from both sides.
<code>build_cross_match()</code>	miss_gst, miss_pb	DataFrame	Finds GSTINs common to both missing sheets. Sums values per GSTIN from each side, shows diff.

4.5 Excel Formatting Functions

Function	Input	Returns	Purpose
<code>_fmt_ws(ws, hex_col)</code>	worksheet, hex colour	None	Applies header fill, alternating row fills, borders, number format, auto column widths, freeze row 1.
<code>_highlight_mismatch()</code>	ws, df	None	Finds _PB vs _GST columns, highlights cells red where values differ

			by more than ₹0.01.
<code>_totals_row(ws)</code>	worksheet	None	Appends a SUM formula row at the bottom for all VALUE_COLS and Diff columns.
<code>_write_vendor_summary_sheet()</code>	ws, df, month	None	Writes vendor summary with colour-coded headers (green=PB, purple=GST, orange=diff).
<code>_write_cross_match_sheet()</code>	ws, df	None	Writes cross match sheet. Shows 'no common GSTINs' message if df is empty.
<code>_write_flat_summary()</code>	ws, matched, mismatched, miss_gst, miss_pb, vendor_df, all_dups	None	Writes Summary sheet — counts table + ITC risk + vendor-wise table below.
<code>export_report()</code>	matched, mismatched, miss_gst, miss_pb, pb_agg, gst_agg, all_dups, out_folder, log	str	Orchestrates all sheet writes. Returns path of saved file.

5. GUI Class — App(tk.Tk)

The entire GUI is one class that extends tkinter's Tk root window. All methods prefixed with underscore are private/internal.

Method	Purpose
<code>__init__()</code>	Sets up state variables (gst_files, pb_files, out_folder, progress), calls <code>_build_ui()</code>
<code>_build_ui()</code>	Builds the entire window layout — header bar, file pickers, run button, progress bar, log box
<code>_multi_file_row()</code>	Creates a reusable card with Listbox + Browse + Clear buttons for multi-file selection
<code>_card()</code>	Helper — creates a styled white card frame with a title label, returns inner frame
<code>_browse_out()</code>	Opens folder selection dialog, updates out_folder StringVar
<code>_log(msg)</code>	Thread-safe log write — schedules log_box insert via <code>self.after(0, ...)</code>
<code>_set_status()</code>	Thread-safe status label update

<code>_set_progress(val)</code>	Thread-safe progress bar update
<code>_watch_count()</code>	Polls file list length every 500ms, updates count label colour
<code>_start()</code>	Validates inputs, clears log, launches <code>_pipeline()</code> in background thread
<code>_pipeline()</code>	Background thread function — calls <code>run_reconciliation()</code> , reads result counts, shows badges + popup

Threading Model

The GUI runs on the main thread. The reconciliation pipeline runs on a background thread.

Communication back to GUI uses `self.after(0, callback)` — the ONLY thread-safe way in tkinter.

`_log()`, `_set_status()`, `_set_progress()` all use `self.after()` internally.

Never call tkinter widgets directly from the pipeline thread — it will crash.

6. How to Add / Change Features

6.1 Add a New Column Name Variant

If a new GST portal format uses a different column name not in the current list:

```
# In COL_ALIASES dict, add the new variant (lowercase):
"new column name here" : "IGST",    # maps to standard IGST column

# Standard names you can map TO:
# "GSTIN", "Invoice Number", "Invoice Date",
# "Taxable Value", "IGST", "CGST", "SGST",
# "ITC Eligibility", "Vendor Name"
```

6.2 Change Mismatch Tolerance

```
MISMATCH_TOL = 5.0    # Was 1.0 — now ₹5 tolerance
```

6.3 Add a New Output Sheet

Example: add a sheet called 'High_Value' for invoices above ₹1 lakh:

```
# Step 1: Create the DataFrame in run_reconciliation()
high_value = matched[matched['Taxable Value_PB'] > 100000].copy()

# Step 2: Add to export_report() call:
path = export_report(matched, mismatched, miss_gst, miss_pb,
                      pb_agg, gst_agg, dups_df, out_folder, log,
                      high_value=high_value)    # add param

# Step 3: Inside export_report(), add to data_sheets list:
data_sheets.append(("High_Value", high_value, "Matched")) # reuse green colour
```

6.4 Re-enable Duplicate Detection

The duplicate detection code is commented out. To re-enable:

```
# In run_reconciliation(), find these commented lines and uncomment:

all_dups = []    # was: # all_dups = []

pb_clean, pb_dups = detect_and_remove_duplicates(pb_raw, 'PB', log)
if not pb_dups.empty:
    pb_dups['Source'] = 'Purchase Book'
    all_dups.append(pb_dups)

# And at export step:
dups_df = pd.concat(all_dups, ignore_index=True) if all_dups else pd.DataFrame()

# In export_report(), uncomment:
if not all_dups.empty:
    data_sheets.append(("Duplicates_Removed", all_dups, "Duplicates_Removed"))
```

6.5 Add Month-wise Split Back

The `group_files_by_month()` function is already written (line 264). To re-enable month-wise reconciliation, replace `run_reconciliation()` with the previous version that calls it. The old version is in git history.

6.6 Change Window Size

```
# In App.__init__():
self.geometry("820x680")    # change width x height
self.resizable(False, False) # change to True to allow resizing
```

7. DataFrame Schemas

These are the exact columns present at each stage of the pipeline:

After `load_gst_file()` / `load_pb_file()`

Raw Loaded DataFrame

GSTIN	— str, cleaned (uppercase, no spaces)
Invoice Number	— str, cleaned (no special chars, uppercase)
Invoice Date	— str (not parsed to datetime yet)
Taxable Value	— float64
IGST	— float64
CGST	— float64
SGST	— float64
ITC Eligibility	— str (GST file only, 'N/A' if missing)
Vendor Name	— str (if column found in source file)

Source Sheet — str (GST file only — which sheet row came from)

After aggregate()

Aggregated DataFrame (one row per invoice)

GSTIN — str
 Invoice Number — str
 Taxable Value — float64 (summed)
 IGST — float64 (summed)
 CGST — float64 (summed)
 SGST — float64 (summed)
 ITC Eligibility — str (worst value across rows for that invoice)
 Vendor Name — str (first value found for that invoice)

After reconcile() — Mismatched sheet

Mismatched DataFrame columns

GSTIN, Invoice Number
 Vendor Name (PB), Vendor Name (GST)
 Taxable Value_PB, IGST_PB, CGST_PB, SGST_PB — from Purchase Book
 Taxable Value_GST, IGST_GST, CGST_GST, SGST_GST — from GST Portal
 Taxable Value Diff, IGST Diff, CGST Diff, SGST Diff
 Total Tax Diff — sum of abs(IGST+CGST+SGST diff), used for mismatch check

8. Building the .exe File

Use PyInstaller to package gstr2a_app.py into a single .exe that works on any Windows machine without Python installed.

One-time setup on your Windows machine:

```
# Make sure Python 3.10+ is installed
# Open Command Prompt and run:
pip install pandas openpyxl pyinstaller
```

Build command:

```
pyinstaller --onefile --windowed --name GSTR2A_Reconciliation_Tool gstr2a_app.py

# --onefile    = everything in one .exe (slower to start but easier to share)
# --windowed   = no black console window behind the GUI
# --name       = name of the output .exe file
```

Output:

```
dist/
  GSTR2A_Reconciliation_Tool.exe ← this is the file to share
build/
  GSTR2A_Reconciliation_Tool.spec ← build config, keep for rebuilds
```

Rebuild after changes:

```
# After editing gstr2a_app.py, just run again:
pyinstaller GSTR2A_Reconciliation_Tool.spec

# This is faster than the full command above
```

Important Notes for Build

Build must be done on Windows to produce a Windows .exe
 Build on Mac produces a Mac .app — different file
 The .exe file size will be ~40-80MB — normal for PyInstaller + pandas
 First launch takes 3-5 seconds — PyInstaller extracts files to temp folder
 Subsequent launches on same machine are faster
 Antivirus may flag it — this is common with PyInstaller, not a real threat

9. Known Limitations & Future Work

Limitation	Details / Suggested Fix
Duplicate detection disabled	Code exists (detect_and_remove_duplicates) but commented out. Needs a UI toggle to enable/disable before re-activating.
CDNR not treated as negative	Credit notes reduce ITC but are currently summed same as invoices. Fix: add a 'Note Type' column and multiply values by -1 for credit notes.
No carry-forward detection	Late-filed invoices appear in next month's GSTR-2A. Tool marks them as missing. Fix: add date-range tolerance matching.
group_files_by_month() unused	Month-split pipeline was replaced with flat pipeline. Function kept for future use.
No progress within a step	Progress bar jumps in fixed increments (8→25→45...). For large files, it appears stuck. Fix: emit progress inside load_gst_file() per sheet.
Window is not resizable	Fixed at 820x680. For very large screens, layout may look small. Fix: change resizable(True,True) and use relative sizing.
Single output filename	Always saves as Final_Reco_Report.xlsx. Previous run gets overwritten. Fix: append timestamp to filename.

10. Quick Reference Card

Key Line Numbers (approximate)

Line 28-66 — Constants: OUTPUT_FILE, KEY_COLS, VALUE_COLS, MISMATCH_TOL, COL_ALIASES

Line 72-92 — Colours: GUI palette (C dict), Excel sheet colours (SHEET_COLOURS)

Line 144-170 — Cleaners: clean_invoice(), clean_gstin(), _get_col(), _rename_cols()

Line 173-215 — Header detection: _find_header_row(), _extract_sheet()

Line 216-260 — File loaders: load_gst_file(), load_pb_file()

Line 308-325 — Aggregation: aggregate()

Line 341-395 — Reconciliation: reconcile()

Line 453-511 — Vendor summary builder: build_vendor_summary()

Line 596-677 — Cross match builder: build_cross_match()

Line 761-829 — Export orchestrator: export_report()

Line 997-1127 — Summary sheet writer: _write_flat_summary()

Line 1128-1200 — Full pipeline: run_reconciliation()

Line 1207-end — GUI class: App(tk.Tk)

Most Likely Places You Will Edit

To add column aliases → COL_ALIASES dict (~line 38)

To change tolerance → MISMATCH_TOL (~line 31)

To add output sheet → export_report() data_sheets list (~line 785)

To change GUI colours → C dict (~line 72)

To change window size → App.__init__() (~line 1210)

To fix a column mapping → _rename_cols() + COL_ALIASES

To change output path → export_report() — OUTPUT_FILE constant

— End of Developer Reference —